GLOBAL SOLUTION 2026.1 · FIAP · ECONOMIA ESPACIAL

OrbiTerra

Inteligência espacial contra o estresse hídrico no café

Satélite + sensor de campo (ESP32) + IA — transformando dado em decisão

---

Grupo: **[nome do grupo]**

Integrantes:

Kaique — RM562072

Willian — RM571013

*Para concorrer ao pódio, inserir aqui a frase: **"QUERO CONCORRER"**.*

Tema: Economia espacial · Observação da Terra (downstream)
Foco da POC: Café arábica — corredor Mogiana Paulista / Sul de Minas
Documento: Entrega final (POC) — Introdução, Desenvolvimento, Resultados Esperados e Conclusões

1. Introdução

O **OrbiTerra** é uma Prova de Conceito (POC) desenvolvida para a **Global Solution 2026.1** da FIAP, cujo tema é a **economia espacial**. A GS pede uma solução que responda: *"Como a tecnologia espacial pode ser utilizada para melhorar a vida das pessoas, tornar processos mais eficientes e criar novas oportunidades na Terra ou fora dela?"*

Nossa resposta parte do maior mercado *downstream* da economia espacial — a **observação da Terra** — e o coloca a serviço de um problema real e mensurável: a **perda de produtividade do café por falta d'água**. O café arábica tem faixa de tolerância climática estreita, e boa parte da produção vem de pequenas e médias propriedades onde a decisão de "quando irrigar" ainda é tomada por intuição. O foco é o corredor cafeeiro da **Mogiana Paulista / Sul de Minas**, próximo à equipe e estratégico para o país.

A ideia central é combinar **três olhares** sobre a lavoura: o **satélite** (visão macro — índices de vegetação NDVI/EVI e temperatura de superfície LST), uma **estação de campo com ESP32** (visão micro — umidade do solo na zona da raiz e temperatura/umidade do ar) e a **Inteligência Artificial** (XGBoost e Random Forest), que funde tudo e classifica o **risco de estresse hídrico dos próximos 7 dias** em três níveis: Baixo, Médio e Alto. O resultado é exibido em um **painel de decisão** que indica onde está o risco e quando agir.

2. Desenvolvimento

2.1 Arquitetura da solução (5 camadas)

A solução tem cinco camadas: (1) **satélite óptico** (Sentinel-2/Landsat via HLS) para NDVI/EVI e LST; (2) **produto térmico** (MODIS LST); (3) **clima** (CHIRPS para chuva, NASA POWER para temperatura); (4) **estação de campo ESP32** (umidade do solo + DHT22), o indicador antecipado; e (5) o **pipeline em Python/Pandas** → **modelo (XGBoost/Random Forest)** → **painel (Streamlit)**. O princípio-chave: o solo reage **antes** do NDVI — por isso o sensor agrega valor preditivo.

Na POC, o **ESP32 é simulado** (o edital permite) e os **dados são sintéticos, mas fisicamente plausíveis**, com semente fixa (100% reproduzível). O caminho futuro é plugar dados reais via Google Earth Engine.

2.2 Geração dos dados e o rótulo de risco

O simulador modela um **balanço hídrico do solo** (modelo tipo "balde"): a água disponível na raiz sobe com a chuva infiltrada e cai com a evapotranspiração (que cresce com o calor). Esse balanço latente é a **verdade** que define o risco — ele é o **alvo**, nunca uma variável de entrada (evitando vazamento de dados). O ESP32 apenas o *observa com ruído*.

```
# src/simulador_dados.py — balanço hídrico do solo (LATENTE: é o ALVO, não feature)
et0 = np.clip(2.4 + 0.16 * (temp_ar - 17.0), 1.2, 6.5)        # demanda diária (ET)
agua = np.empty(N_DIAS); agua[0] = 72.0
for t in range(1, N_DIAS):
    recarga = 0.9 * max(0.0, (40.0 - agua[t-1]) / 40.0)       # capilaridade do subsolo
    novo = agua[t-1] + 0.85 * precip[t] + recarga - et0[t]    # entra chuva, sai ET
    agua[t] = float(np.clip(novo, 0.0, 100.0))

# Sensor ESP32: OBSERVA a água do solo com ruido (indicador ANTECIPADO)
umidade_solo = np.clip(agua + rng.normal(0, 6.5, N_DIAS), 0, 100)
```

O **rótulo** é o risco dos próximos 7 dias, definido pela menor umidade do solo na janela futura — o "pior momento que vem":

```
# src/simulador_dados.py — rótulo: risco nos PRÓXIMOS 7 dias (3 classes)
# Olhamos a MENOR água do solo na janela futura (o pior momento que vem).
agua_min_futura = (pd.Series(agua).shift(-1)
                   .rolling(7, min_periods=7).min().to_numpy())
```

```
risco[agua_min_futura < 28] = 2                                  # ALTO
risco[(agua_min_futura >= 28) & (agua_min_futura < 58)] = 1   # MEDIO
risco[agua_min_futura >= 58] = 0                                 # BAIXO
```

2.3 Engenharia de variáveis e treino dos modelos

Com Pandas, criamos variáveis derivadas que dão **memória** e **tendência** às séries — chuva acumulada, dias sem chuva, média e tendência de 3 dias da umidade do solo (o sinal de alerta precoce) e sazonalidade. Nada usa o futuro.

```
# src/treino_modelo.py — engenharia de variáveis (nada usa o futuro)
# Sinais do ESP32: nivel, media recente e TENDENCIA de 3 dias (alerta precoce!)
df["umidade_solo_7d"]     = df["umidade_solo"].rolling(7, min_periods=1).mean()
df["umidade_solo_tend_3d"] = df["umidade_solo"].diff(3).fillna(0.0)
# Clima acumulado (memoria de chuva) e estiagem
df["precip_7d"]  = df["precip_mm"].rolling(7,  min_periods=1).sum()
df["precip_30d"] = df["precip_mm"].rolling(30, min_periods=1).sum()
```

Treinamos **Random Forest** e **XGBoost** em dois cenários — **Modelo A** (satélite + clima) e **Modelo B** (A + ESP32) — com **validação temporal**: treina nos primeiros 75% do tempo e testa nos 25% finais, sem nunca embaralhar as datas. Assim medimos honestamente quanto o sensor de campo agrega.

```
# src/treino_modelo.py — dois algoritmos, validacao TEMPORAL (treina passado, testa futuro)
modelos = {
  "Random Forest": RandomForestClassifier(n_estimators=300, max_depth=12,
                     min_samples_leaf=5, class_weight="balanced", random_state=42),
  "XGBoost":       XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05,
                     subsample=0.9, colsample_bytree=0.9, num_class=3),
}
corte = int(len(df) * 0.75)               # 75% do tempo para treino, 25% final para teste
treino, teste = df.iloc[:corte], df.iloc[corte:]
# Cenarios:  A = satelite + clima   |   B = A + ESP32  -> comparamos A x B
```

2.4 A estação ESP32 (simulada) e o painel

O emulador faz o papel do firmware: gera leituras dos sensores em “tempo real”, aplica um **alerta na borda** (mesmo offline a estação já avisa) e transmite cada leitura em **JSON**, como a placa enviaria por WiFi (HTTP/MQTT). O painel (Streamlit) consome os dados e o modelo para mostrar mapa, risco previsto, a série NDVI × umidade do solo e a recomendação de manejo.

```
# src/emulador_esp32.py — firmware do ESP32 SIMULADO (alerta na borda + JSON via WiFi)
def nivel_alerta(umidade_solo):
    if umidade_solo < 28:  return "ALTO"
    if umidade_solo < 58:  return "MEDIO"
    return "BAIXO"

# Cada leitura sai como um JSON, igual a placa enviaria por HTTP/MQTT:
# {"id_estacao": "ESP32-MOGIANA-01", "umidade_solo": 31.4,
#  "temp_ar": 21.0, "umidade_ar": 68.6, "alerta_local": "ALTO"}
```

3. Resultados Esperados

O experimento confirma a tese: **adicionar o ESP32 eleva a previsão** em ambos os algoritmos, com validação temporal. A tabela compara o Modelo A (satélite + clima) com o Modelo B (+ESP32):

Algoritmo	Modelo A (satélite+clima)	Modelo B (+ESP32)	Ganho
Random Forest	83,9%	97,4%	+13,6 pt
XGBoost	78,8%	93,0%	+14,3 pt
Baseline (classe maj.)	68,9%	—	—

A **variável mais importante** do Modelo B é a umidade do solo do ESP32 (`umidade_solo_7d` ≈ 0,51, seguida de `umidade_solo`). Mais do que acurácia, o ganho é de **segurança**: o Modelo A **subestima** o risco — classifica 49 dias de risco Médio como Baixo, o que deixaria o produtor sem irrigar — e o **Modelo B corrige** esse erro perigoso (ver matriz de confusão).

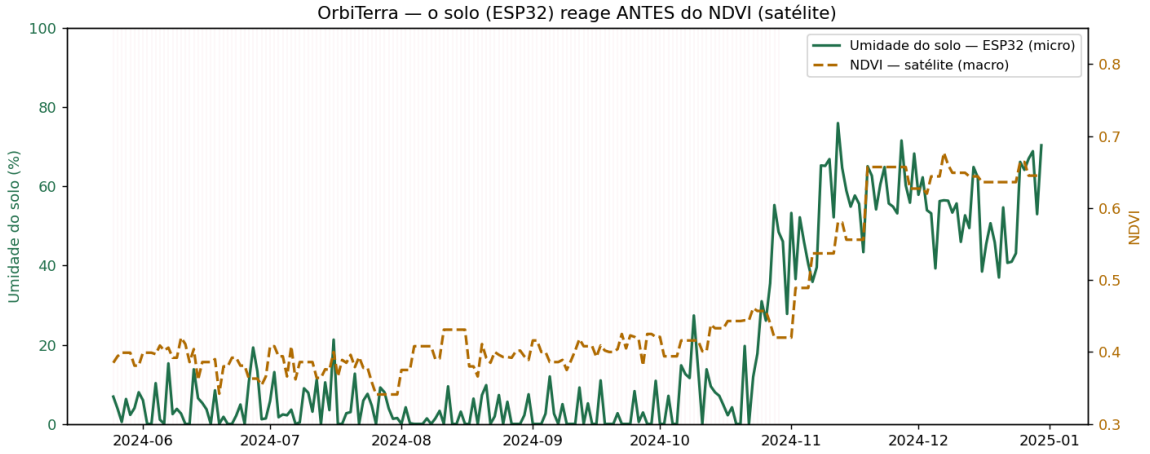


Fig. 1 — A umidade do solo (ESP32) cai ANTES do NDVI (satélite), que é defasado e tem lacunas de nuvem. Essa antecedência gera o alerta precoce.

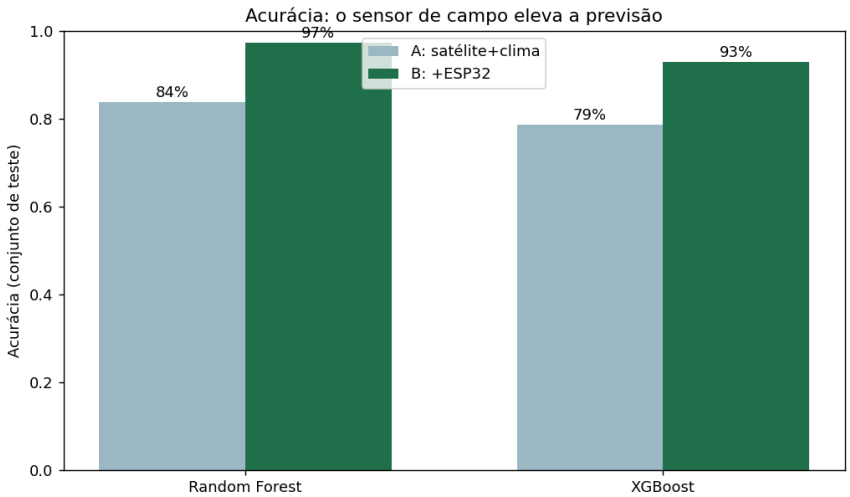


Fig. 2 — Acurácia no teste: o sensor de campo eleva a previsão nos dois algoritmos.

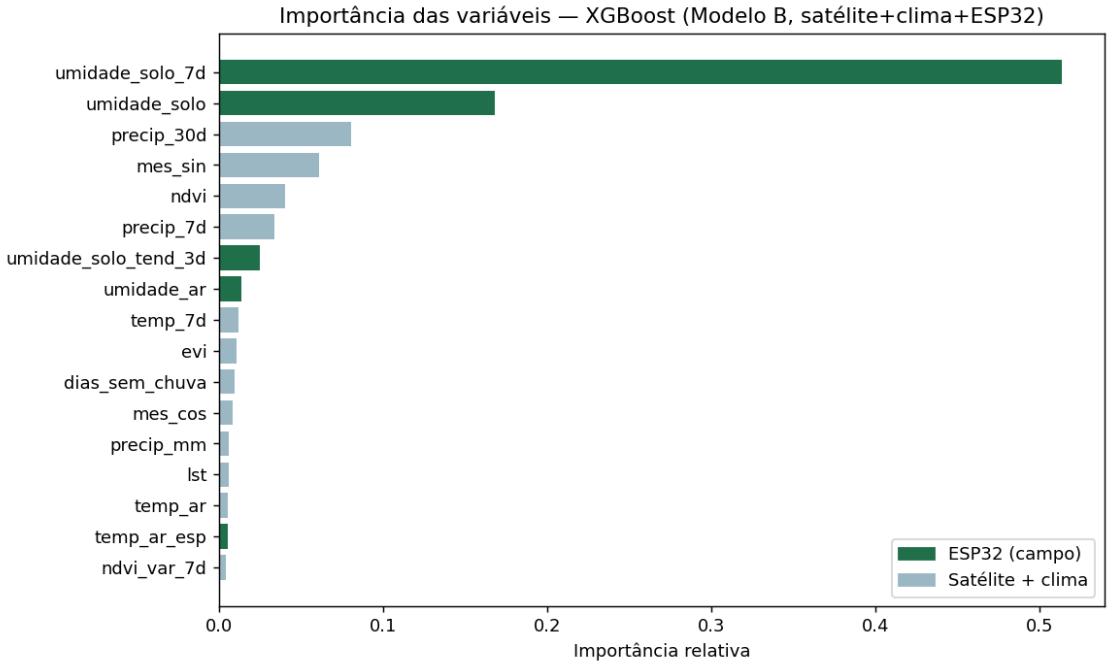


Fig. 3 — Importância das variáveis (XGBoost, Modelo B): os sinais do ESP32 (verde) dominam o topo.

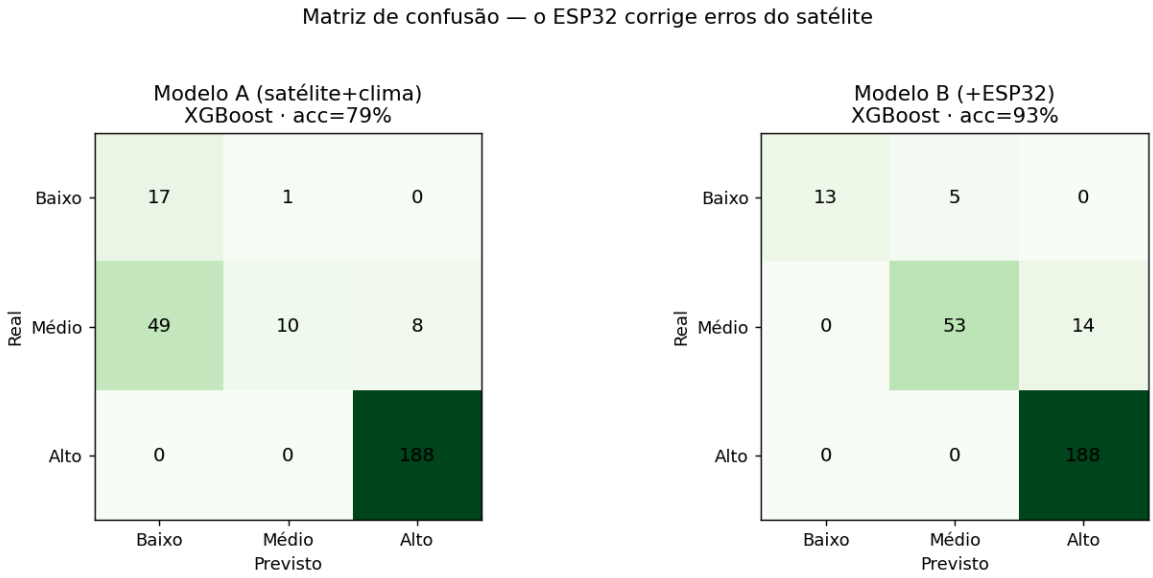


Fig. 4 — Matriz de confusão: o Modelo A troca “Médio” por “Baixo” (subestima o risco); o Modelo B corrige.

4. Conclusões

O OrbiTerra demonstra, de ponta a ponta, como a **tecnologia espacial** pode virar **decisão prática** no campo. A POC integra dados de satélite, clima e um sensor de campo (ESP32) e prova, com um experimento controlado e validação temporal, que **o sensor de campo se paga**: ele eleva a acurácia (RF +13,6 pt; XGB +14,3 pt) e, sobretudo, corrige o erro mais perigoso — subestimar o risco hídrico.

A solução responde às três partes da pergunta da GS: **melhora a vida das pessoas** (protege a renda do cafeicultor e o preço ao consumidor), **torna processos mais eficientes** (irrigar na hora certa, sem desperdício) e **cria oportunidades** (um serviço de inteligência agrícola replicável e de baixo custo, que pode alimentar crédito rural e seguro paramétrico).

Como POC, assumimos limites com transparência: ESP32 simulado e dados sintéticos (porém plausíveis e reproduzíveis), sem vazamento de dados. A evolução natural é substituir o simulador por dados reais via Google Earth Engine (Sentinel-2/MODIS/CHIRPS) e instalar a placa física no talhão — o código já está preparado para receber leituras reais via HTTP/MQTT.

Integração de disciplinas (Fases 3–4): Lógica e Python; Análise de dados (Pandas); Visualização (Matplotlib e painel Streamlit); Machine Learning (XGBoost e Random Forest); IoT/embarcados (ESP32); e Banco de dados/APIs (JSON e CSV).

Links da entrega

• **Repositório do projeto:** [inserir link do repositório aqui]

• **Vídeo demonstrativo (YouTube, “Não listado”):** [inserir link do vídeo aqui]

• **Documento de conceito:** docs/OrbiTerra_Conceito.pdf · **Arquitetura/fluxograma:** docs/arquitetura.md · **Roteiro do vídeo:** docs/roteiro_video.md